

NAME: CHHEANG HOKIENG

ID::e20220695

GROUP : I4_GIC_C

▣ TP06 Project

All requirements from **TP06.pdf** have been successfully implemented and tested!

1. Access the Backend

URL: <http://localhost:8100>

2. Login Credentials

Role	Email	Password
Admin	admin@example.com	password
Manager	manager@example.com	password
Staff 1	staff1@example.com	password
Staff 2	staff2@example.com	password

localhost:8100/login



Welcome Back

Please sign in to your account

Email Address

These credentials do not match our records.

Password

☐ Remember me [Forgot password?](#)

[Sign In](#)

Don't have an account? [Sign up for free](#)

3. Features Available

As Admin

- View/manage ALL projects and tasks
- Manage users (full access)
- Manage products and categories
- Complete system control
- Can logout from top-right dropdown

As Manager

- Create and manage YOUR projects
- Create tasks and assign to staff
- View tasks in your projects
- Manage products and categories
- Can logout from top-right dropdown

As Staff

- View tasks assigned to YOU
- Update status of your tasks
- View related project details
- Can logout from top-right dropdown ""

What's Been Implemented

Part 0 - Starter Setup

- ☑ Laravel Breeze installed
- ☑ Login/Register working
- ☑ Session authentication

Part 1 - Database Design for RBAC

- ☑ **roles** table (admin/manager/staff)
- ☑ **permissions** table
- ☑ **permission_role** pivot table
- ☑ **role_user** pivot table
- ☑ **projects** table

- ☑ `tasks` table
- ☑ All models with relationships

Part 2 - Seed Data

- ☑ 1 Admin user
- ☑ 1 Manager user
- ☑ 2 Staff users
- ☑ Permissions (users.manage, products., **categories.**)
- ☑ 3 Sample projects
- ☑ 6 Sample tasks assigned to staff

Part 3 - Authorization with Gates

- ☑ `Gate::before()` for admin bypass
- ☑ Gates for users.manage
- ☑ Gates for products.* (create, update, delete, view)
- ☑ Gates for categories.* (create, update, delete, view)
- ☑ Helper methods: `hasRole()`, `hasPermission()`

Part 4 - Policies

- ☑ `TaskPolicy` with `view()` and `updateStatus()`
- ☑ `CategoryPolicy` with `view()` and `updateStatus()`
- ☑ Policies registered in `AuthServiceProvider`
- ☑ Used in controllers with `authorize()`

Part 5 - Laravel Passport (API Auth)

- ☑ Passport installed and configured
- ☑ API guard set to 'passport'
- ☑ POST `/api/login` returns access token
- ☑ GET `/api/me` returns user + roles
- ☑ Protected API routes with `auth:api` middleware
- ☑ Role-based API authorization ""

Backend Features

Dashboard (<http://localhost:8100/dashboard>)

- Role-based welcome message
- Quick links to Projects and Tasks
- Permission summary for your role
- API access information

[Dashboard Screenshot]

Projects Management (<http://localhost:8100/projects>)

- **Admin/Manager:** Create, edit, delete projects
- **Staff:** View projects they have tasks in
- Project status: active, completed, on_hold
- Shows task count per project

Tasks Management (<http://localhost:8100/tasks>)

- **Admin/Manager:** Create, edit, delete tasks
- **Staff:** View and update status of their tasks
- Task statuses: pending, in_progress, completed
- Assign tasks to staff members

[Tasks Management Screenshot]

Logout Functionality

HOW TO LOGOUT: 1. Click your **name** in the top-right corner 2. Select "**Log Out**" from dropdown 3. You'll be redirected to the home page

Note: JavaScript assets have been compiled. If logout doesn't work: - Hard refresh browser (Ctrl+F5) - Clear browser cache - See [LOGOUT_GUIDE.md](#) for troubleshooting

API Testing

1. Login to get token

```
curl -X POST http://localhost:8100/api/login \  
  -H "Content-Type: application/json" \  
  -H "Accept: application/json" \  
  -d '{
```

```
"email": "admin@example.com",  
"password": "password"  
}'
```

Response:

```
{
  "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1b290bm9udGVudCI6ImFkbWluX290bm9udGVudCIsImV4cCI6MTY1OTU0MDAwfQ",
  "user": {
    "id": 1,
    "name": "Admin User",
    "email": "admin@example.com"
  }
}
```

2. Get current user with roles

```
curl -X GET http://localhost:8100/api/me \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Accept: application/json"
```

3. Get tasks (role-based filtering)

```
curl -X GET http://localhost:8100/api/tasks \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Accept: application/json"
```

4. Update task status (staff only their tasks)

```
curl -X PATCH http://localhost:8100/api/tasks/1/status \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -d '{"status": "in_progress"}'
```

5. Create product (manager/admin only)

```
curl -X POST http://localhost:8100/api/products \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -d '{
```

```
"name": "New Product",  
"category_id": 1,  
"pricing": 99.99,  
"description": "Product description"  
}'
```

Project Structure

Key Files Created/Modified

Models

- `app/Models/User.php` - User with roles & permissions
- `app/Models/Role.php` - Roles (admin/manager/staff)
- `app/Models/Permission.php` - Permissions
- `app/Models/Project.php` - Projects (NEW)
- `app/Models/Task.php` - Tasks (FIXED)

Controllers

- `app/Http/Controllers/ProjectController.php` - Project CRUD (NEW)
- `app/Http/Controllers/TaskController.php` - Task CRUD (NEW)
- `app/Http/Controllers/ProductController.php` - With authorization (UPDATED)
- `app/Http/Controllers/CategoryController.php` - Full CRUD (UPDATED)

Policies

- `app/Policies/TaskPolicy.php` - Task authorization
- `app/Policies/CategoryPolicy.php` - Category authorization

Views

- `resources/views/dashboard.blade.php` - Role-based dashboard (UPDATED)
- `resources/views/projects/index.blade.php` - Projects list (NEW)
- `resources/views/tasks/index.blade.php` - Tasks list (NEW)

Routes

- `routes/web.php` - Web routes with project/task resources (UPDATED)
- `routes/api.php` - API routes with Passport auth (UPDATED)

Migrations

- `database/migrations/*_create_projects_table.php` (NEW)
- `database/migrations/*_create_tasks_table.php` (NEW)

Seeders

- `database/seeders/ProjectSeeder.php` (NEW)
- `database/seeders/TaskSeeder.php` (NEW)
- `database/seeders/DatabaseSeeder.php` (UPDATED) ""

☐☐ Useful Commands

Database

```
# Reset database with fresh data
docker-compose exec app php artisan migrate:fresh --seed

# Run only seeders
docker-compose exec app php artisan db:seed
```

Cache

```
# Clear all caches
docker-compose exec app php artisan optimize:clear

# Clear specific caches
docker-compose exec app php artisan config:clear
docker-compose exec app php artisan view:clear
docker-compose exec app php artisan route:clear
```

Assets

```
# Build for production (already done)
docker-compose exec app npm run build

# Watch for changes (development)
docker-compose exec app npm run dev
```

Routes

```
# List all routes
docker-compose exec app php artisan route:list

# List API routes only
docker-compose exec app php artisan route:list --path=api

# List web routes only
docker-compose exec app php artisan route:list --path=/
```

Passport

```
# Create new personal access client
docker-compose exec app php artisan passport:client --personal
```

□ Testing Checklist

Admin Testing

- ☒ Login as admin
- ☒ View all projects
- ☒ Create new project
- ☒ View all tasks
- ☒ Create task and assign to staff
- ☒ Update any task
- ☒ Delete project/task
- ☒ Logout successfully

Manager Testing

- ☒ Login as manager
- ☒ View only your projects
- ☒ Create new project
- ☒ View tasks in your projects
- ☒ Assign task to staff
- ☒ Cannot see other manager's projects

- ☑ Logout successfully

Staff Testing

- ☑ Login as staff
- ☑ View only assigned tasks
- ☑ Update task status
- ☑ Cannot create projects
- ☑ Cannot assign tasks
- ☑ Cannot see other staff's tasks
- ☑ Logout successfully

API Testing

- ☑ Login via API (all roles)
- ☑ Get user info via /api/me
- ☑ Get tasks (role-filtered)
- ☑ Update task status (authorized)
- ☑ Create product (manager/admin only)
- ☑ 403 error for unauthorized actions ""

☐ Submission Ready

This project fulfills **ALL** requirements from **TP06.pdf**:

Requirement	Status
Authentication (Breeze)	☐ Complete
Authorization (Gates)	☐ Complete
RBAC (Roles & Permissions)	☐ Complete
Policies (Object-level)	☐ Complete
API Auth (Passport)	☐ Complete
Project Tracker	☐ Complete
Database Design	☐ Complete
Seeders	☐ Complete
Controllers with Auth	☐ Complete
Web Interface	☐ Complete